Leonardo: an integrated PyMOL-based platform for structural bioinformatics workflow orchestration and verification

Juvenal Yosa[1,2*], Matthew R. Groves[3] and Anthonie J. van der Wekken[2]

[1*]Protyon Tech, Colombia.
[2]University Medical Center Groningen, Groningen, The Netherlands.
[3]Chemical and Pharmaceutical Biology, Groningen Research Institute of Pharmacy, University of Groningen, Groningen, The Netherlands.

*Corresponding author(s). E-mail(s): jyosa@protyon.tech;

Abstract

Background: Structural bioinformatics increasingly combines sequence annotation, experimental or predicted structures, ligand preparation, docking, molecular dynamics diagnostics and quantum chemistry. These steps are usually distributed across command-line programs, notebooks and molecular viewers, creating a barrier for experimental or interdisciplinary users who need repeatable computational experiments but do not necessarily have computational-specialist training. **Results:** We present Leonardo, a modular PyMOL-based application that organizes these tasks into project-aware workflows. Leonardo distinguishes adapter modules that expose established tools from native workflow pipelines that add object mapping, workflow checks, repeated sampling and reaction-path construction. The architecture covers 15 modules and more than 30 cited external tool or service dependencies. Representative module-level examples exercise protein-ligand contact reporting, fpocket/smina pocket-to-grid handoff, single-target template redocking, ligand-library quality control, OpenMM collective-variable setup, guided-exit binding diagnostics and Psi4/MOPAC reaction-path orchestration. These examples document that Leonardo can prepare inputs, preserve object provenance, execute or parse external engines, generate PyMOL-linked reports and rebuild manuscript figures from traceable source artifacts. The software verification suite completed with 526 passing tests in the declared `protliginteract-full` environment. The reported examples are workflow-verification cases rather than general benchmarks of docking, virtual-screening enrichment, production binding free energies, force-field accuracy or

transition-state energetics. **Conclusions:** Leonardo is designed to reduce manual handoffs between biological inputs and physics-based or cheminformatics analyses while keeping specialized algorithms transparently cited and externally replaceable. Its contribution is a reproducible structural bioinformatics workflow layer rather than a replacement for established scientific engines.

Keywords: structural bioinformatics, protein-ligand interactions, molecular docking, molecular dynamics, PyMOL, workflow software

1 Background

Protein structure analysis has moved from a specialist computational activity to a routine part of many biological projects. Experimental structures, AlphaFold models, mutagenesis hypotheses, ligand libraries, enzymatic mechanisms and molecular-dynamics trajectories are now commonly interpreted together. A single study may begin with sequence retrieval and motif interpretation, continue through homology modeling or structure prediction, require ligand sourcing and protonation checks, proceed to docking or virtual screening, and then use molecular dynamics diagnostics, formal free-energy methods or quantum chemistry for mechanistic questions. Mature tools exist for each layer, including Biopython for biological sequence and structure parsing [1], RDKit and Open Babel for cheminformatics [2, 3], AutoDock Vina and smina for docking [4, 5], fpocket for pocket detection [6], OpenMM for molecular simulation [7], MBAR-based methods for free-energy analysis [8], MODELLER for comparative modeling [9], Psi4 and MOPAC for quantum chemistry [10, 11], and AlphaFold/ColabFold for protein structure prediction [12, 13].

The practical barrier is that these methods are rarely experienced by wet-lab biologists, structural biologists, biochemists or medicinal chemists as one coherent experiment. Many powerful engines are distributed as command-line programs, Python libraries, environment-specific notebooks or separate graphical programs. A user may retrieve a sequence from UniProt, align homologues with BLAST+ or HMMER, generate an AlphaFold or MODELLER model, inspect it in a molecular viewer, prepare a ligand with one tool, dock it with another, run molecular dynamics in a separate environment and finally generate the analysis figures with custom code [14–18]. Each transition requires file conversion, atom naming consistency, object selection, parameter transfer and interpretation of method-specific output. For users who are not trained as computational scientists, these handoff steps can become the main obstacle to using simulations as repeatable biological experiments rather than as isolated expert services.

This usability issue has scientific consequences. Errors at workflow boundaries can change biological conclusions: an incorrect ligand protonation state or formal charge can alter salt-bridge interpretation, an ambiguous receptor object can invalidate a docking run, an unconstrained unbinding coordinate can let a ligand leave through an unphysical route, and a missing cofactor, ion or structural water can produce a model that no longer represents the experimental system. Reproducibility is affected

in the same way. If a published result depends on undocumented manual selections, hidden file edits, custom local code or a sequence of graphical actions that cannot be reconstructed, then repeating the computational experiment becomes difficult even when the underlying algorithms are public.

Commercial molecular-modeling suites demonstrate the value of integrated environments. Schrödinger Maestro is presented as a unified graphical entry point for molecular discovery workflows, and BioLuminate extends the same ecosystem toward biologics and protein engineering [19, 20]. The Molecular Operating Environment (MOE) is positioned as an integrated computer-aided molecular design platform for small molecules, peptides and biologics [21]. BIOVIA Discovery Studio brings modeling and simulation tools for life-science research into a common environment [22]. CCDC GOLD provides a well-established protein-ligand docking workflow within the CSD-Discovery/CSD-Enterprise ecosystem [23]. These licensed platforms are important because they show what many scientists need: guided setup, visual inspection, linked analysis panels, standard workflows and reduced friction between modeling steps. However, cost, licensing, local installation constraints and limited transparency of some proprietary components can restrict adoption, especially in small laboratories, teaching environments and groups that need auditable open workflows.

Open-source and academically available tools address complementary needs. PyMOL, UCSF Chimera/ChimeraX and VMD provide widely used molecular visualization and analysis environments [24–27]; PyMod illustrates how PyMOL can host sequence-structure and homology-modeling workflows [28]. Open-source scientific engines and libraries such as NumPy, Matplotlib, RDKit, Open Babel, Meeko, OpenMM, OpenFF, MDTraj, AutoDock Vina, smina, fpocket, Psi4, MOPAC, Biopython and ColabFold provide high-quality computational capabilities across numerical analysis, scientific plotting, cheminformatics, docking, molecular simulation, quantum chemistry and structure prediction [1–7, 10, 11, 13, 29–33]. The remaining gap is not the absence of algorithms but the absence of a lightweight, modular, PyMOL-native suite that helps non-computational users assemble these algorithms into repeatable structural bioinformatics experiments while preserving explicit citations and external-tool boundaries.

Table 1 summarizes Leonardo's position relative to these related software classes. The comparison is functional rather than performance-based: Leonardo does not attempt to outscore docking engines, replace molecular viewers or duplicate commercial suites. Its specific contribution is to connect project objects, engine handoffs, result parsing and manuscript-level traceability in one PyMOL-centered workflow layer.

Leonardo was developed for this gap. The goal is to let a user start from biological objects that are familiar in a molecular viewer—a sequence, a protein structure, a ligand, a binding site, a trajectory or a cluster of atoms—and move through analysis with guided windows, documented defaults, visible object choices, background jobs, summaries and standardized figures. In this sense Leonardo is intended to be enabling software for non-specialist users: it gives them access to docking, redocking, virtual screening, interaction analysis, protein modeling, mutation analysis, sequence readiness checks, OpenMM collective-variable simulations, guided-exit binding diagnostics and quantum/semiempirical reaction-path workflows without requiring every step to

Table 1 Functional comparison with related structural-bioinformatics software classes.

Software class	Representative examples	Primary strength	Leonardo distinction
Commercial molecular-modeling suites	Maestro/BioLuminate, MOE, Discovery Studio, GOLD [19–23]	Mature integrated modeling environments with extensive supported workflows	Open-source, PyMOL-native, auditable workflow layer with explicit third-party boundaries, reproducible manuscript artifacts and inspectable Python modules
Molecular viewers	PyMOL, ChimeraX, VMD [24, 26, 27]	Visualization, scene generation and interactive molecular inspection	Adds project-aware docking, redocking, sequence, OpenMM diagnostic, guided-exit, reaction-path and report workflows around viewer objects
Focused PyMOL plugins	PyMod [28]	Sequence-structure and comparative-modeling workflows inside PyMOL	Broader module set spanning ligands, docking, 2D-to-3D interaction maps, task monitoring, OpenMM diagnostics and Psi4/MOPAC reaction-path orchestration
Interaction profilers	PLIP/ProLIF-style contact detection [34, 35]	Automated protein-ligand contact annotation	Conservative PyMOL-linked contact reporting with charge guardrails and two-dimensional map selection; three-complex ProLIF count comparison is included, while curated atom-pair benchmarking remains future work
Command-line scientific engines	RDKit, smina, fpocket, OpenMM, Psi4, MOPAC, ColabFold [2, 5–7, 10, 11, 13]	Specialized algorithms and numerical methods	Object selection, parameter handoff, repeated-seed bookkeeping, task tracking, result parsing and reproducible figure/report generation

be manually scripted. At the same time, it keeps the computational responsibility transparent: Leonardo records the workflow and prepares, runs or parses established engines rather than claiming to replace them.

Leonardo is evaluated here as a structural bioinformatics and biological data-analysis platform whose contribution is reproducible integration: maintainable workflows, graphical object selection, project state management, background task monitoring, parameter handoff, result inspection and plotting around established tools. Its purpose is to make computational structural biology more usable and repeatable for

experimental and interdisciplinary groups, while remaining modular enough for computational developers to inspect, extend and replace individual engines as methods evolve.

2 Implementation

2.1 Design goals and software architecture

Leonardo is implemented as a modular Python application centered on PyMOL and Qt. The main application layer manages molecular objects, user selections, background tasks, logs, project metadata and report generation. Scientific functionality is exposed through runtime modules for structure acquisition, interaction analysis, redocking, pocket docking, virtual screening, ligand sourcing and parametrization, protein modeling and mutation, sequence analysis, OpenMM dynamics, guided-exit binding diagnostics, trajectory interaction analysis, MOPAC capped-cluster workflows, optional OpenMM/MOPAC QM/MM helper routines, Psi4 quantum workflows, AlphaFold/ColabFold preparation and task monitoring. This separation keeps user-interface code distinct from scientific backends and allows individual runtime modules to be tested without launching the full graphical application.

The central design goal is not to replace established engines. Leonardo treats PyMOL objects, sequence records, ligand files, trajectories and reaction clusters as project entities, then prepares inputs for specialized programs, records parameters, parses outputs and returns interpretable molecular scenes, tables and figures. This organization is intended for users who can state the biological question—for example whether a mutation removes a binding interaction, whether a ligand can be redocked into an experimental pose, whether an alanine-dipeptide collective variable was sampled, or whether a capped reaction cluster has a plausible transition state—but who should not need to hand-edit every intermediate file.

The software boundary is implemented through runtime modules with explicit input and output contracts, coordinated by the graphical controller. A module receives project objects or files, validates required selections, writes an execution payload, starts either a local parser or an external executable, and returns structured tables, logs and optional PyMOL scene updates. Background jobs are registered with the task monitor so that command, input object names, working directory, exit status and parsed error messages remain inspectable after completion or failure. Generated tables are used as the primary provenance layer for figures: figures are rebuilt from CSV, JSON, XML or PyMOL script artifacts rather than from manual drawing. This structure lets developers replace an external engine or add a new runtime module with limited changes to the graphical application, provided the module preserves the same project-state and artifact contracts.

2.2 Adapter modules around established software

Several Leonardo modules are adapter workflows: they expose established software through a more coherent PyMOL-aware interface, but the underlying scientific method remains the responsibility of the external program. This distinction is important

because it prevents over-claiming. For structure acquisition, Leonardo fetches and labels structures from the Protein Data Bank and predicted-structure resources, then keeps the source and object identity visible in downstream windows [12, 36]. For sequence bioinformatics, Leonardo retrieves UniProt annotations, summarizes motifs and prepares alignment or search handoffs to BLAST+, HMMER, MAFFT and Clustal Omega [14–18]. For comparative modeling and mutation analysis, Leonardo prepares user selections and model inspection around MODELLER and Biopython [1, 9]. For ligand preparation, docking and virtual screening, Leonardo uses RDKit, Open Babel, Meeko, AmberTools/OpenFF, fpocket and smina while adding object selection, ligand-library checks, job tracking and pose inspection [2, 3, 5, 6, 31, 32, 37]. For structure prediction, Leonardo prepares local or Google Colab ColabFold/AlphaFold runs and imports the resulting models back into the same project [12, 13].

In practice these adapter modules support common biological use cases. A laboratory studying a new enzyme can fetch a homologous PDB structure, annotate conserved sequence motifs, build a mutant model and inspect the active site before planning mutagenesis. A medicinal-chemistry group can import vendor SDF libraries, standardize structures, detect failed molecules and launch a pocket-focused docking screen while preserving the receptor and ligand object names used to generate the grid. A structural biologist without a solved structure can transfer a sequence and ColabFold parameters into a reproducible notebook rather than manually copying FASTA text into an empty Colab session. In each case, Leonardo's contribution is the workflow context, not a new BLAST, docking, modeling or structure-prediction algorithm.

2.3 Workflow pipelines implemented in Leonardo

Other modules add workflow logic that is implemented inside Leonardo and then combined with external engines. The interaction-analysis pipeline performs conservative contact typing, charge-aware salt-bridge checks, residue-level grouping and 2D-to-3D selection handoff. The redocking pipeline aligns a crystal-ligand template, runs repeated docking seeds and reports both docking affinity and template RMSD. The OpenMM dynamics pipeline exposes predefined collective variables, umbrella/metadynamics setup and diagnostic-grid plotting for users who do not write OpenMM scripts directly. The guided-exit binding diagnostic module, historically named ABFE-lite in the user interface and codebase, adds resource-bounded execution limits, staged analysis checks and a guided cylindrical ligand-exit restraint to reduce unphysical exit paths. The Psi4 and MOPAC reaction modules implement Python-side QST-like synchronous-transit guess generation, transition-state checks, frequency checks and IRC path plotting, while Psi4 or MOPAC provides the quantum or semiempirical energy model. The QST-like workflow is motivated by synchronous-transit/quasi-Newton transition-state search concepts described by Peng and Schlegel, and the IRC profile is interpreted in mass-weighted coordinates following Gonzalez and Schlegel [38, 39].

Table 2 defines the boundary between Leonardo workflows and the external engines they orchestrate. This boundary is used throughout the manuscript to separate three

claims: external programs provide specialized algorithms, Leonardo provides project-aware orchestration and several modules provide native workflow logic around those programs.

Table 2 Leonardo module boundary used for manuscript framing.

Module	Leonardo workflow contribution	External engines, services or libraries
Structure acquisition	Fetch/import workflow, object naming, source tracking and PyMOL handoff	RCSB PDB, AlphaFold DB, ESMFold/Meta, PyMOL
Sequence bioinformatics	UniProt motif summaries, readiness checks, alignment handoff and pop-up reports	UniProt, BLAST+, HMMER, MAFFT, Clustal Omega, Biopython
Protein modeling and mutation	Mutation setup, repair workflow, template handoff and model inspection	MODELLER, Biopython, PyMOL
Interaction analysis	Contact typing, charge guardrails, salt-bridge checks and 2D-to-3D selection	PyMOL, NumPy, Biopython
Ligand source and parametrization	Ligand registry, editing, format validation, charge checks and parameter handoff	RDKit, Open Babel, AmberTools, OpenFF, Psi4
Pocket docking	Object-aware receptor/ligand selection, pocket/grid definition and pose analysis	fpocket, smina, Meeko, RDKit
Redocking	Template mapping, repeated seeds, RMSD recovery and score/RMSD summaries	smina, RDKit, Meeko
Virtual screening	Batch ligand preparation, job tracking, ranking and hit handoff	RDKit, Meeko, smina
OpenMM dynamics	Protocol wizard, predefined CVs, umbrella/metadynamics setup and diagnostic-grid plotting	OpenMM, OpenFF, AmberTools, PyMBAR, MDTraj
Trajectory interaction analysis	Contact occupancy, distance/RMSD summaries and report generation	MDTraj, NumPy, Matplotlib
Guided-exit binding diagnostic (historical ABFE-lite module)	Resource-constrained restraint design, guided ligand-exit path and state-gating diagnostics	OpenMM, PyMBAR, OpenFF
MOPAC capped-cluster reaction	QM cluster selection, caps, QST-like workflow labels mapped to MOPAC SADDLE/TS/IRC and PM7 path plots	MOPAC, NumPy, Matplotlib
Psi4 quantum	Python QST-like guess generation, TS/frequency/IRC workflow orchestration and plots	Psi4, NumPy, Matplotlib
AlphaFold workflows	Local/Colab preparation, sequence export, parameter transfer and result import	AlphaFold, ColabFold, Google Colab
Task monitor and reports	Background-task registry, log parsing, error summaries and metadata export	Python, PyTest

2.4 Workflow-verification design and reproducibility

The verification strategy separates workflow correctness from scientific benchmarking. Unit and backend tests exercise parsers, object selection, charge guardrails, task-monitor state handling and runtime payload construction. The distributed manuscript workflow regenerates figures and numerical summaries from curated source artifacts; this is figure-and-report reproducibility, not a claim that every external calculation is rerun end-to-end during manuscript rebuild. Where full reruns are possible, the source package records raw inputs, command templates, seeds, engine outputs or parsed logs so reviewers can distinguish computational reruns from figure regeneration. These examples demonstrate that Leonardo prepares inputs, calls or parses external engines,

preserves object provenance and renders interpretable outputs; they are not presented as exhaustive benchmarks of the underlying docking, molecular-dynamics, ABFE, quantum-chemistry or structure-prediction algorithms.

All reported numerical summaries and figures were regenerated from source artifacts using the manuscript workflow distributed with the source release. The release includes installer documentation, environment specifications, test reports, source-data tables and checksums for generated outputs. Table 3 records the computational environment used for the workflow examples reported here, and Table 4 summarizes how manuscript claims map to source artifacts. The full traceability table is distributed as `draft/results/reproducibility_traceability.csv`; the companion file `draft/results/workflow_reproduction_commands.csv` separates local test execution, external-engine rerun templates and figure-rebuild commands.

Table 3 Computational environment used for workflow verification.

Component	Version or configuration
Python	3.11.14
NumPy / Matplotlib	NumPy 1.26.4; Matplotlib 3.10.8
RDKit / Biopython	RDKit 2023.09.6; Biopython 1.86
OpenMM / MDTraj / PyMBAR	OpenMM 8.4; MDTraj 1.11.1; PyMBAR 4.2.0
Docking and quantum engines	smina 2020.12.10; Psi4 1.9.1; MOPAC 23.2.3
Operating system and hardware	macOS 26.0 (build 25A354), CPU-only workflow generation; GPU acceleration was not required for the reported examples
Reproducibility files	`environment.yml` SHA-256 prefix 27e0aa0be7515ff0; macOS environment export `environment-protliginteract-full.lock.yml` SHA-256 prefix 14dd0266c88352fb; `pyproject.toml` SHA-256 prefix b6376e70b2f57f12; figure script SHA-256 prefix 6464b1c882c74b73

3 Results and workflow verification

3.1 Architecture map and module boundary

Leonardo integrates biological inputs, workflow modules, external engines and scientific outputs in a single project-aware interface (Figure 1). The classification includes 15 modules and explicitly separates Leonardo's native contribution from third-party algorithms. This boundary is central to the software claim: Leonardo contributes the reproducible workflow layer, not reimplementations of the external engines. The architecture is organized into four layers: biological inputs, Leonardo workflow pipelines, cited external engines and reproducible outputs.

Table 4 Claim-versus-evidence traceability for representative manuscript examples. The complete machine-readable version is supplied as `draft/results/reproducibility_traceability.csv`.

Manuscript item	Evidence artifact	Rebuilt output	Claim boundary
Architecture and workflow-feature matrix	Pipeline classification and inventory CSV tables	Figures 1–4	Manual software categorization, not performance measurement
Interaction analysis	PDB 3PTB and Leonardo contact table	Figures 6 and 7	Single-system contact-report workflow demonstration
Pocket docking and redocking	fpocket/smina outputs and 1STP template-redocking tables	Figures 5 and 8	Handoff, parsing and RMSD-reporting checks, not multi-target docking benchmark
Ligand library and OpenMM CVs	Local SDF descriptor table and ACE-ALA-NME CV diagnostic tables	Figures 9 and 10	Library quality-control and compact-molecule plotting checks
Guided-exit diagnostic and reaction paths	Sanitized guided-exit diagnostic JSON/CSV/report tables and QST/IRC summary tables	Figures 11, 12 and 13	Diagnostic workflow and parsing checks, not production ABFE or general TS benchmark
Regression tests	PyTest XML report	Workflow-verification summary JSON	Software regression evidence, not scientific-method benchmarking

The architecture is exposed to users through a single workflow launcher that keeps structure-loading controls, ligand-state summaries, interaction controls, module entry points and task-monitoring utilities visible in one place (Figure 2). This interface is intended to make the suite usable as a coherent structural-bioinformatics workspace rather than as a collection of disconnected scripts.

The module comparison identified six pipeline groups: inputs/project state, sequence/structure preparation, protein-ligand modeling, simulation/free-energy analysis, quantum/reaction paths and prediction/external compute. Across the 15 modules Leonardo orchestrates more than 30 cited external tool or service dependencies, with dependency breadth ranging from two to six tools per module. Workflow-support categories include automated tests, workflow checks, case demonstrations, report/guardrail

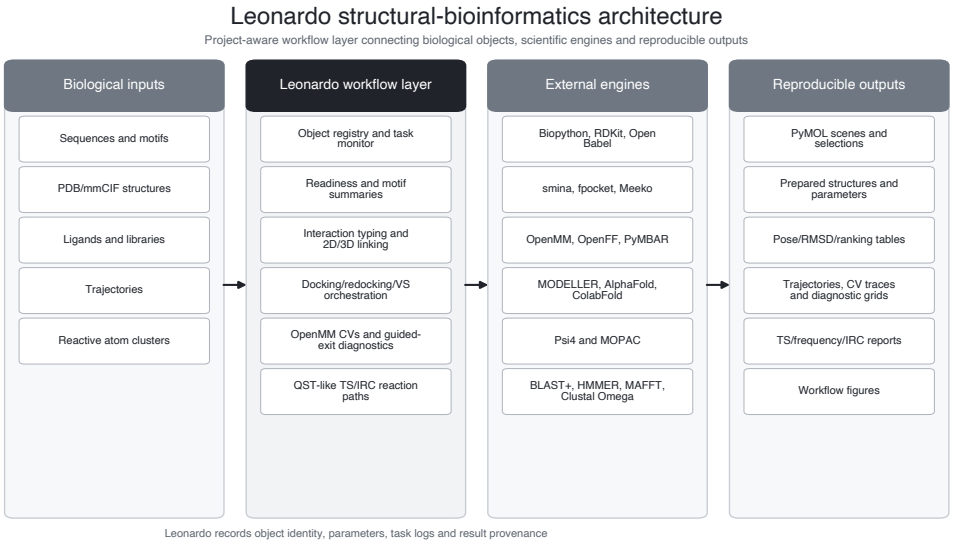


Fig. 1 Leonardo workflow architecture. Biological inputs are managed by Leonardo workflows, which call specialized external engines and return PyMOL scenes, prepared structures, tables, reports and figures. The contribution boundary is the project-aware GUI, object mapping, workflow verification, reporting and plotting layer; specialized engines provide the cited numerical methods.

checks and rendered figures (Figure 3). These categories document implemented software surfaces and available checks; they do not assign equal scientific validation strength to every module.

The module workflow-feature matrix provides a more granular view of these comparisons (Figure 4). Adapter modules such as AlphaFold workflows and pocket docking are represented by object handoff, workflow execution or report generation. Native pipelines such as interaction analysis, redocking, OpenMM dynamics and the Psi4/MOPAC reaction modules show additional support artifacts for plotting, case examples or repeated sampling. The matrix is an inventory of implemented workflow surfaces and available checks; it is not intended to rank scientific validation strength across modules.

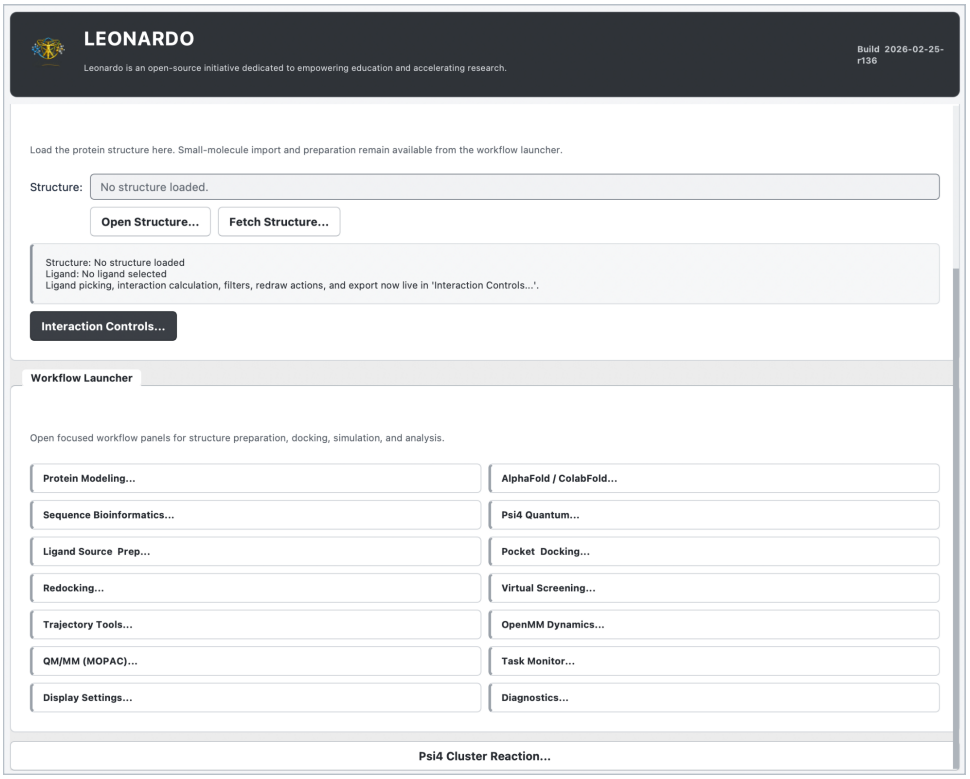


Fig. 2 Leonardo suite interface. The main window provides structure loading and fetching, ligand and interaction-state summaries, interaction controls, workflow-launch buttons for the major modules and access to task monitoring and diagnostics. The screenshot documents the user-facing integration layer that connects PyMOL objects to Leonardo workflow modules.

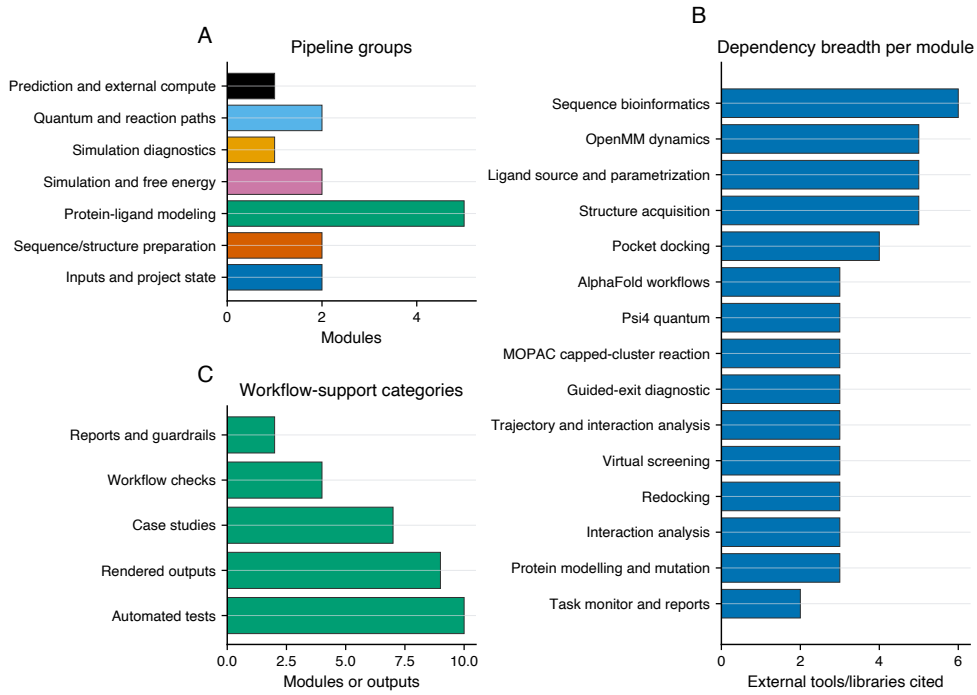


Fig. 3 Module inventory for $n = 15$ Leonardo modules. (A) Module counts by workflow group. (B) Number of cited external tools or services per module. (C) Workflow-support categories represented in tests, workflow checks, case demonstrations, reports and rendered outputs. Counts are manual workflow categories rather than independent scientific performance measurements.

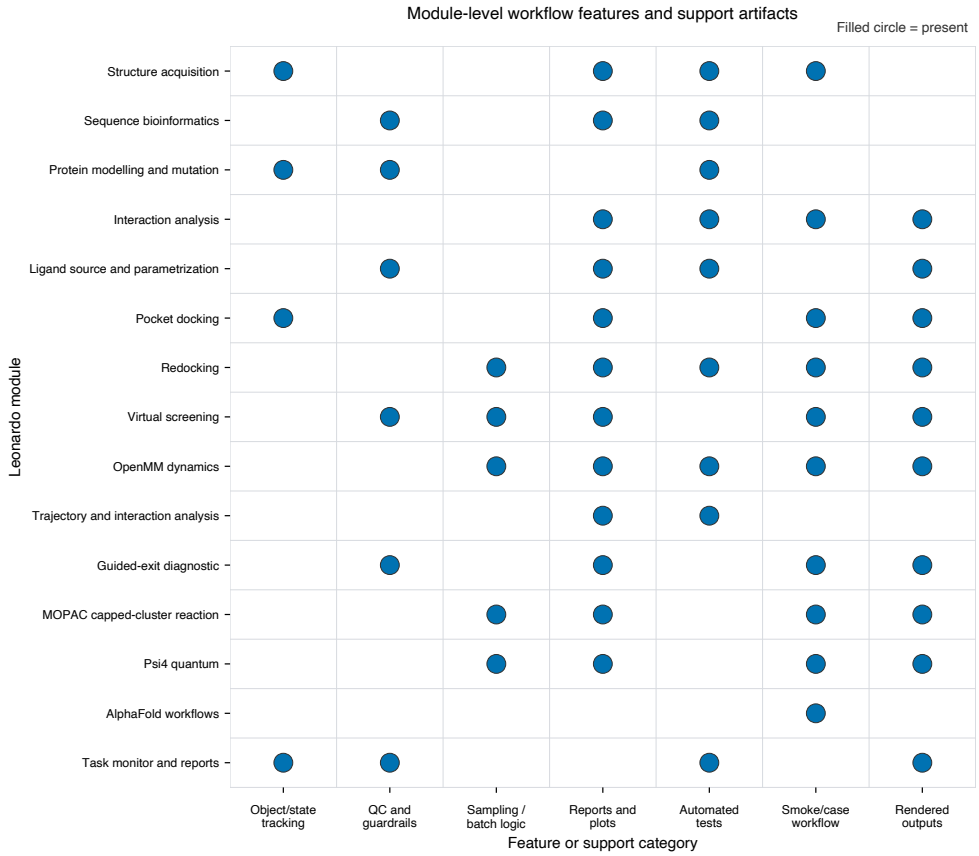


Fig. 4 Module-level workflow features and support artifacts. A filled circle indicates that the module contains the corresponding workflow capability, workflow check, case example, automated test or rendered output. Symbols do not encode scientific benchmark strength or compare accuracy across modules.

3.2 Modules that expose established tools

3.2.1 Structure acquisition

The structure-acquisition module is an adapter around public structural resources and PyMOL object creation. It fetches experimental structures from the Protein Data Bank and imports predicted or local models while preserving source, chain and object names for downstream modules [36]. Leonardo does not implement a new structure-prediction or structure-archive algorithm in this module; its purpose is to prevent ambiguous receptor objects and undocumented file handoffs. A typical use is to load an apo enzyme, a holo PDB structure and a predicted model into the same PyMOL project before selecting the receptor used for docking or mutation analysis.

3.2.2 AlphaFold and ColabFold workflows

The AlphaFold module prepares local or Google Colab ColabFold runs by transferring the working sequence, job name and selected parameters into a runnable prediction workflow [12, 13]. The scientific prediction is performed by AlphaFold/ColabFold, not by Leonardo. Leonardo contributes sequence export, parameter transfer, result import and model inspection inside the same project. This is useful when a user has a UniProt sequence but no experimental structure, or when a predicted domain model is needed before ligand-pocket inspection.

3.2.3 Bioinformatics sequence analysis

The sequence-analysis module connects sequence records to UniProt motif summaries, readiness reports and alignment/search handoffs to BLAST+, HMMER, MAFFT and Clustal Omega [14–18]. This module does not replace those algorithms. Its role is to bring motif positions, residue numbers and sequence context into pop-up summaries that can guide structural work. For example, a user can identify a catalytic motif, map its residues onto a model and then decide which positions should be included in mutation modeling or interaction analysis.

3.2.4 Protein modeling and mutation

Protein modeling and mutation is a guided adapter around MODELLER, Biopython and PyMOL inspection [1, 9, 24]. Leonardo prepares mutation choices, template/model handoff, repair checks and scene updates, while comparative modeling and structural parsing are performed by the external tools. The module is intended for biological questions such as whether a resistance mutation is near a binding site, whether a missing loop should be modelled before docking, or whether a mutation disrupts a salt bridge seen in the interaction-analysis module.

3.2.5 Ligand source and parametrization

The ligand-source and parametrization module manages chemical inputs before docking, simulation or quantum calculations. It uses RDKit and Open Babel for parsing and conversion, and prepares handoffs to Meeko, AmberTools, OpenFF or quantum-derived charge workflows where appropriate [2, 3, 31, 32, 37]. Leonardo does not claim

a new force-field or protonation algorithm here. Its value is ligand registry management, format checks, charge/protonation warnings and consistent downstream object selection. In practice, this module helps detect failed SDF records, suspicious formal charges or inconsistent ligand identifiers before a screening or MD job is launched.

3.2.6 Pocket docking

Pocket docking is an adapter workflow around pocket detection, docking-grid definition and docking execution with tools such as fpocket, Meeko and smina [5, 6, 31]. The scoring and pose search are performed by the docking engine. Leonardo contributes object-aware receptor/ligand selection, pocket-to-grid handoff and pose visualization so the user knows exactly which protein and ligand objects were used. The workflow check reported here evaluates object handoff and report generation for the pocket-docking workflow; it does not assess pocket-detection accuracy, docking pose accuracy or scoring performance. A typical use is to compare a crystallographic pocket with an automatically detected pocket before docking a focused inhibitor series.

The pocket-docking workflow example parsed fpocket output with 10 detected pockets, selected pocket 4 by the configured ranking criterion, constructed a docking grid centered at $(8.94, 6.52, 8.50)$ Å with dimensions $(11.43, 16.36, 11.56)$ Å and parsed three smina pose modes for the selected pocket. To address crystallographic-site recovery separately from the smoke workflow, fpocket was also run on the 1STP receptor after removing biotin. The recovered pocket with the smallest ligand-to-alpha-sphere distance was pocket 1; all 16 biotin heavy atoms were within 4 Å of an alpha sphere and the median atom-to-alpha-sphere distance was 0.90 Å. Figure 5 summarizes the pocket ranking, pocket physicochemical descriptors, selected-pocket/grid geometry and docking-mode affinity/RMSD information, while the 1STP/biotin pocket-overlap values are provided in `draft/results/pocket_1stp_crystal_overlap.csv`. These checks support Leonardo's pocket-to-grid and pose-summary reporting path; they are not a broad benchmark of pocket detection or docking scoring.

3.2.7 Task monitor and reports

The task monitor is a support module rather than a scientific algorithm. It records background jobs, opens logs, summarizes failures and keeps workflow metadata accessible. Its purpose is practical reproducibility: a failed smina, OpenMM, Psi4, MOPAC or ColabFold run should produce a readable error summary rather than a silent GUI failure. This module is useful in teaching and shared-lab settings where users need to know which step failed and which input object or executable caused the problem.

3.3 Scientific workflow pipelines implemented in Leonardo

3.3.1 Protein-ligand interaction analysis

The interaction-analysis module is a Leonardo workflow pipeline because contact classification, residue grouping, charge guardrails and 2D-to-3D selection are implemented in Leonardo. The pipeline reduces atom-level observations to residue-level records. For

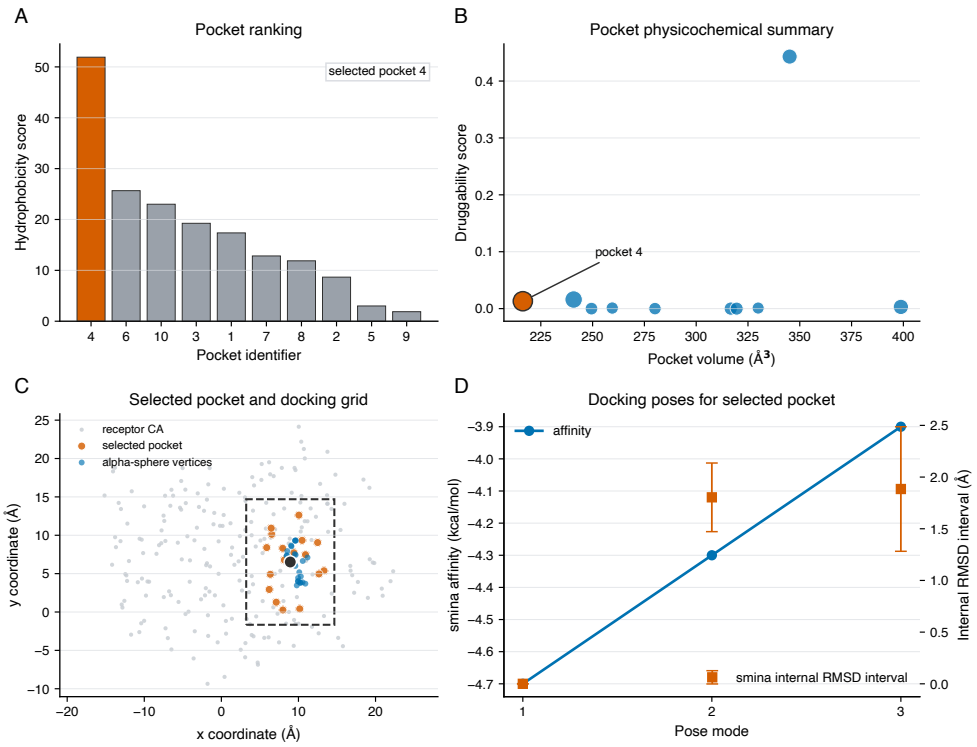


Fig. 5 Pocket-docking workflow check using fpocket and smina outputs. The example parsed $n = 10$ fpocket pockets, selected pocket 4 using the configured ranking rule and constructed the docking grid from that selected pocket. (A) Pocket ranking by hydrophobicity score with the selected pocket highlighted. (B) Pocket volume versus druggability score, with marker size proportional to the number of alpha spheres. (C) Two-dimensional projection of the receptor, selected pocket alpha-sphere/atom records and generated docking grid. (D) smina affinity and the internal lower/upper-bound RMSD interval reported by smina for the parsed pose modes; this is not a crystal-template RMSD. A separate 1STP/biotin crystal-site overlap check recovered fpocket pocket 1 with 100% of biotin heavy atoms within 4 Å of an alpha sphere. The figure checks handoff and reporting, not pocket-detection or docking accuracy.

ligand atoms L and receptor-residue atoms R_r, the closest residue-ligand distance is

$$d_{\min}(r, L) = \min_{i \in R_r,\, j \in L} \|\mathbf{x}_i - \mathbf{x}_j\|_2 \,. \tag{1}$$

Candidate contacts are retained only when distance, element type, residue context and interaction-specific geometry agree. Salt bridges require opposite formal-charge evidence and charged-center proximity; nitrogen atoms are not treated as cations solely because they are nitrogen. This conservative rule is important because bad ligand protonation, missing hydrogens or wrong formal charges would otherwise create false electrostatic interactions.

The 3PTB benzamidine case contained 1629 protein atoms across 223 residues and nine ligand heavy atoms. The reproducible contact analysis used Leonardo's shared

runtime criteria for charge-group, hydrogen-bond fallback, hydrophobic and van der Waals checks, then reduced atom-level contacts to residue-level records. The resulting table contained 15 residue-level contacts: one side-chain hydrophobic contact, four polar contacts, one salt-bridge candidate and nine non-directional close contacts. As an external-profiler check, ProLIF 2.2.0 was run on three local crystal complexes: 3PTB/BEN, 1STP/BTN and 1HSG/MK1 [35]. The comparison is stored as broad interaction-class counts rather than atom-pair truth labels; Leonardo retained 15, 27 and 33 residue-level contacts for the three systems, while ProLIF reported 4, 8 and 10 stricter residue-interaction records, respectively. Count-overlap coefficients were 0.267, 0.296 and 0.303, reflecting the expected difference between Leonardo's conservative retained-contact reporting and ProLIF's more specific interaction fingerprints. This case checks parsing, residue-level grouping, conservative contact labeling for the interaction classes observed in the examples and plot generation. It does not constitute a curated benchmark against atom-pair interaction annotations, and interaction classes absent or rare in these structures still require separate future comparison against curated complexes. The implemented framework also contains rules for hydrogen bonds, halogen bonds, aromatic interactions, cation-pi contacts, metal contacts and hydrophobic/van der Waals clusters, following principles used in protein-ligand interaction profilers [34, 35]; those classes are reported here as implemented contact-reporting capability, not as fully benchmarked chemistry or a claim of precise interaction classification across all protein-ligand systems. The module is used to ask whether a ligand analog preserves key binding contacts, whether a mutation removes a salt bridge or whether a docking pose reproduces interactions seen in a reference structure.

3.3.2 Redocking

The redocking module is a Leonardo workflow pipeline around smina docking. The docking search and affinity scoring are performed by smina, while Leonardo carries a crystallographic template ligand through preparation, runs repeated seeds, parses all modes and reports both docking score and template recovery. For the manuscript script, receptor and template coordinates are prepared in a common crystal frame before docking, and template recovery is reported as no-fit heavy-atom RMSD using the order-preserving heavy-atom correspondence generated from the same Open Babel ligand template:

$$\mathrm{RMSD}_{\mathrm{nofit}} = \sqrt{\frac{1}{N}\sum_{i=1}^{N}\left\|\mathbf{x}_i - \mathbf{y}_{\pi(i)}\right\|^2}, \tag{2}$$

where π is the retained atom correspondence for this prepared template/pose pair. The reported value is therefore appropriate for the 1STP/biotin workflow check but is not presented as a symmetry-corrected ligand-RMSD benchmark for arbitrary chemotypes. In workflows that require receptor alignment before redocking, Leonardo stores the alignment transform separately as a preparation quality-control step; the final ligand recovery value is only reported when an explicit reference ligand is available.

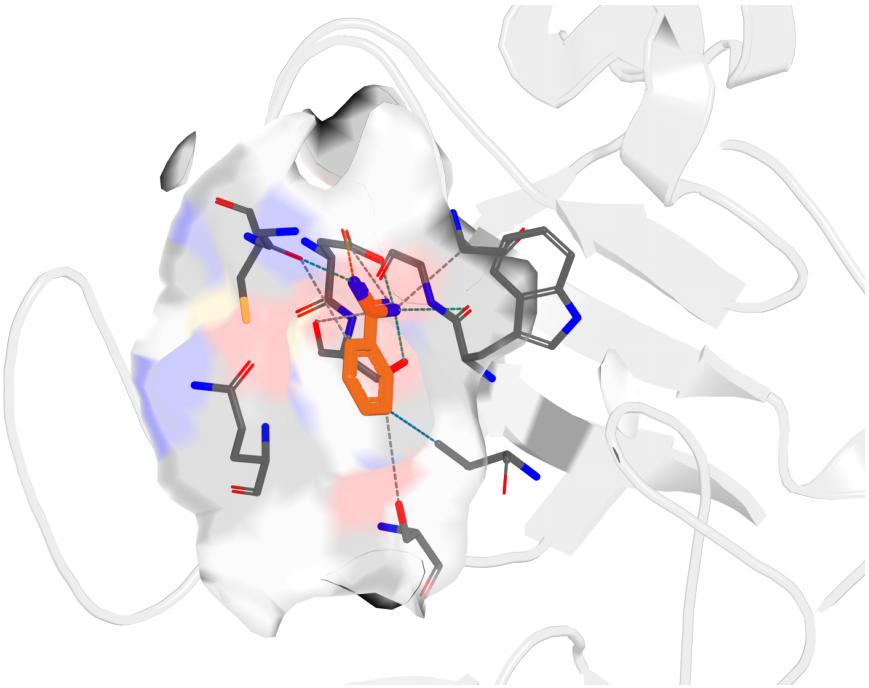


Fig. 6 Protein-ligand interaction scene generated by Leonardo for the 3PTB/benzamidine case. The PyMOL-rendered image shows the ligand in the trypsin binding site, retained contact residues as sticks, a semi-transparent pocket surface and color-coded dashed contact-distance objects derived from Leonardo's contact table. Dashed contacts use the same class colors as Figure 7: polar contacts in green, the salt-bridge candidate in orange, hydrophobic contacts in blue and non-directional close contacts in gray. The scene is intended as a visual interaction report for inspection and communication; quantitative contact counts and distances are summarized separately in Figure 7.

In a single 1STP/biotin redocking case using an explicit crystallographic template and known binding-site grid, 15 seed runs produced 225 pose rows. Each seed generated 15 smina modes, and every seed recovered at least one pose below the common 2.0 Å heavy-atom RMSD success threshold. Across the 15-seed run, the top-scored pose median RMSD was 0.363 Å, the best-pose median RMSD was 0.358 Å, and the minimum observed RMSD was 0.331 Å. These seeds probe workflow reproducibility and stochastic docking variation for one target; they are not independent target-level benchmark replicates. This demonstrates receptor/ligand preparation, seed repetition, preserved-correspondence RMSD reporting and smina-output parsing for this case; it does not test smina scoring accuracy, flexible-receptor treatment or redocking performance across targets. The plot also shows why Leonardo displays both docking score and RMSD-derived recovery metrics when a template ligand is available.

3.3.3 Virtual screening

The virtual-screening module is a Leonardo batch-docking and ligand-library workflow around RDKit, Meeko and smina. Leonardo does not introduce a new scoring function;

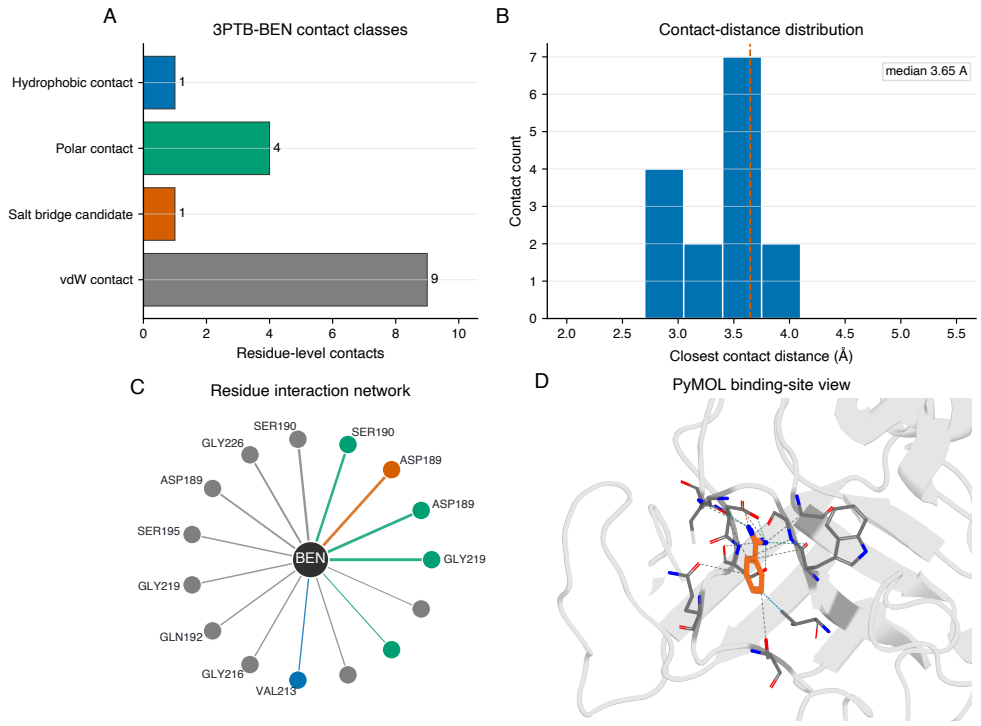


Fig. 7 3PTB benzamidine contact case demonstration. The case contains $n = 15$ retained residue-level contacts from PDB 3PTB and ligand BEN using the workflow input charge/protonation state and conservative charge-aware typing. (A) Residue-level contact counts by conservative class. (B) Closest-distance distribution for retained contacts. (C) Residue-level interaction network linking BEN to retained contact residues; repeated residue labels indicate multiple retained atom-level contacts collapsed to residue-level summaries. (D) PyMOL-rendered binding-site view showing BEN, retained contact residues and dashed contact-distance objects generated from the same contact table. The figure summarizes observed retained contacts and is not a curated benchmark of absent interaction classes.

the default rank is the ordered docking affinity returned by the docking engine,

$$\mathrm{rank}(i) = \mathrm{order}\left(E_i^{\mathrm{dock}}\right), \tag{3}$$

with lower predicted binding energy ranked first. Leonardo adds the pieces that make the run reproducible for non-specialist users: ligand-library parsing, receptor/ligand object selection, batch job creation, failure reporting, hit-table generation and PyMOL handoff of selected poses.

The ligand-library quality-control demonstration used RDKit to parse 369 molecules across 15 curated local SDF files named by target. In addition to molecular descriptors, the workflow records parsed versus failed SDF records, formal-charge flags, multi-fragment records, unassigned stereocenters and duplicate canonical SMILES counts before downstream docking preparation. The median molecular weight was 388.5 Da and the median calculated logP was 3.39. The target-level ligand counts,

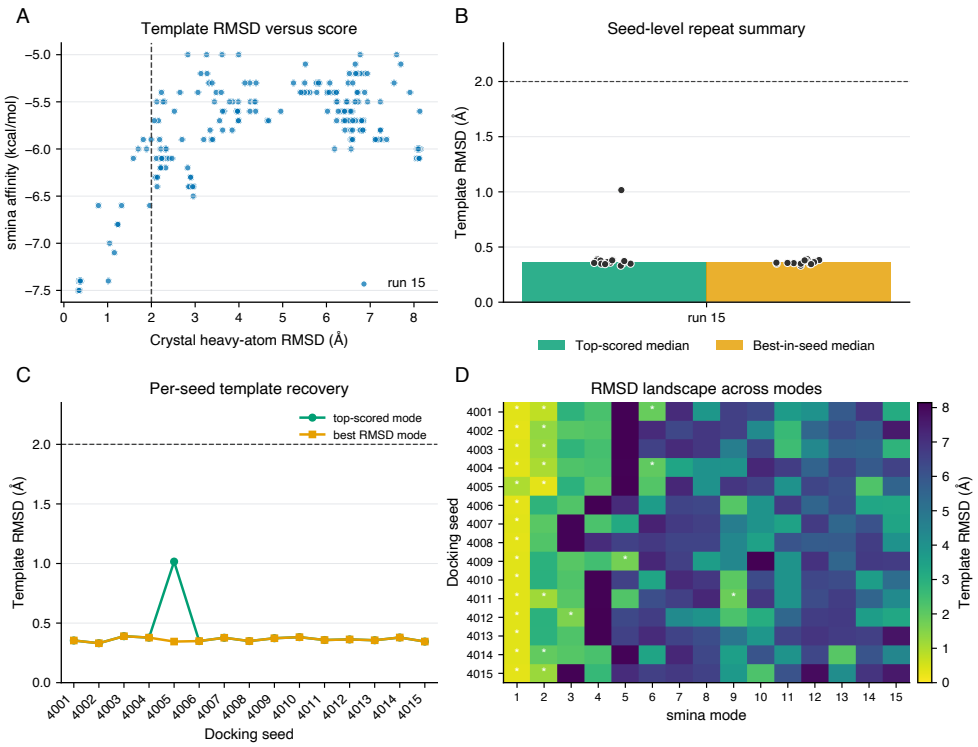


Fig. 8 Single-target 1STP biotin template-redocking workflow check. The case contains $n = 225$ pose rows from $n = 15$ seed runs, with 15 smina modes per seed. (A) Template heavy-atom RMSD versus smina affinity for all modes; the dashed line marks 2.0 Å. (B) Median top-scored and best-per-seed template RMSD with individual seed-level points overlaid. (C) Per-seed template recovery for top-scored and best-RMSD modes. (D) RMSD landscape across smina modes and seeds; asterisks mark modes below 2.0 Å. This checks Leonardo's template mapping and reporting for this case, not general docking accuracy.

molecular-property panel and machine-readable QC table provide a quality-control layer before virtual screening, helping users detect empty libraries, failed SDF parsing, charge/stereo issues or chemical-space mismatches before launching docking jobs. This is not a virtual-screening enrichment benchmark: no actives/decoys, ROC-AUC, enrichment factor, BEDROC, retrospective ranking or prospective hit validation is reported here, and such metrics require appropriate benchmark design [40]. In practical use, this module supports triage of focused inhibitor libraries, comparison of analog series and selection of candidate poses for interaction analysis or follow-up MD.

3.3.4 OpenMM dynamics, collective variables and diagnostic grids

The OpenMM dynamics module is a Leonardo setup and analysis pipeline around OpenMM, OpenFF, AmberTools, PyMBAR and MDTraj [7, 8, 32, 33, 37]. Leonardo prepares protocol stages, exposes predefined collective variables and plots collective-variable traces and diagnostic metadynamics-grid outputs. The module can prepare

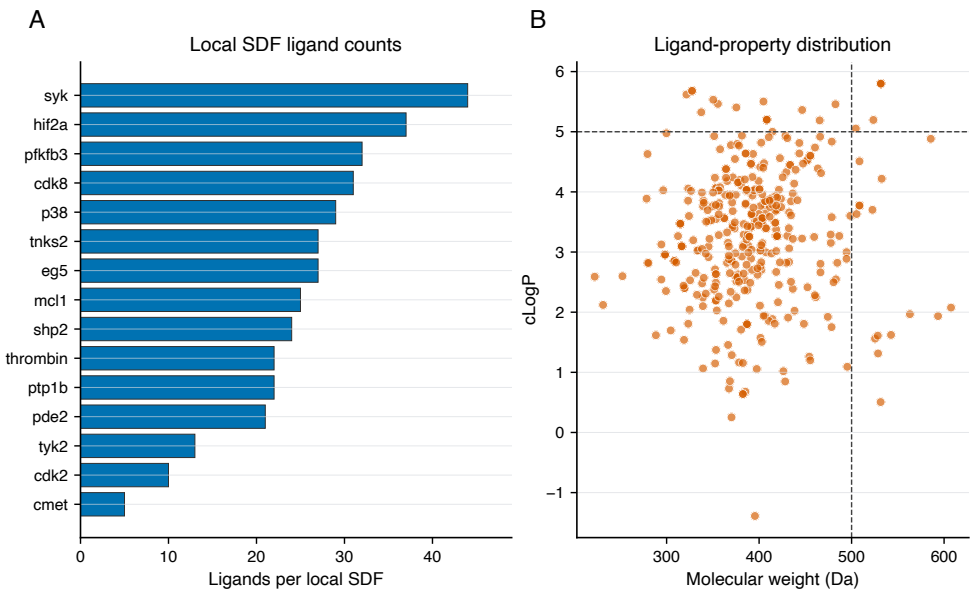


Fig. 9 Local ligand-library quality-control demonstration. RDKit parsed $n = 369$ molecules across 15 local SDF target files. (A) Ligand counts per target. (B) Molecular weight and calculated logP distribution; dashed lines at 500 Da and logP 5 are heuristic QC references, not pass/fail criteria or virtual-screening enrichment evidence.

umbrella or metadynamics-style workflows using established enhanced-sampling concepts [41–43]; the manuscript example exercises setup and plotting rather than comparing enhanced-sampling algorithms. In the alanine example, Leonardo reads OpenMM metadynamics grid values and displays a shifted diagnostic surface,

$$G_{\mathrm{plot}}(\phi,\psi) = G_{\mathrm{OpenMM}}(\phi,\psi) - \min_{\phi,\psi} G_{\mathrm{OpenMM}}(\phi,\psi), \tag{4}$$

with clipping used only for visualization. This display is not an unbiased histogram PMF, WHAM/MBAR estimate or uncertainty-calibrated thermodynamic free-energy surface.

The OpenMM workflow example used ACE-ALA-NME to exercise Leonardo's collective-variable and diagnostic-grid plotting machinery on a compact CPU-compatible system. The phi/psi run produced 2304 grid rows and passed Leonardo's internal plotting-stability criterion after 43,000 steps: the RMSD between consecutive shifted grids remained below 0.5 kJ mol^{-1} for five consecutive blocks. This is a backend and plotting criterion, not a claim of physically converged alanine thermodynamics. A companion CV trace panel exercised eight Leonardo CV presets on the same alanine-dipeptide model, with representative distance, RMSD, torsion and angle traces normalized to z-scores for plotting. This example demonstrates setup, metadynamics-style execution, plotting-stability tracking and figure generation on a small molecule; it is not intended as a biological protein-scale free-energy benchmark or an uncertainty-calibrated force-field study. In real workflows this module is used

to prepare minimization/heating/equilibration protocols, explore torsional sampling, test umbrella or metadynamics CVs and inspect sampling stability before running larger protein systems.

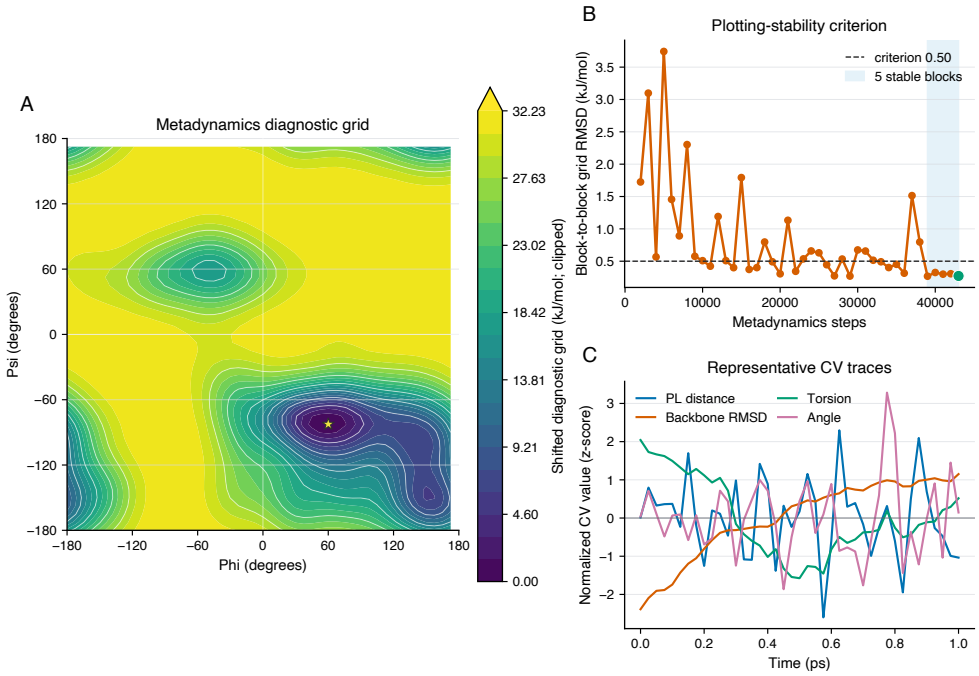


Fig. 10 OpenMM/CV backend check on ACE-ALA-NME. The phi/psi metadynamics-style run used 2304 diagnostic grid rows and passed Leonardo's internal plotting-stability criterion after 43,000 steps. (A) Shifted metadynamics diagnostic grid with the global minimum marked; the displayed color scale is clipped at the 97.5th percentile only for visualization. (B) Block-to-block shifted-grid RMSD trace; the shaded region marks the final five consecutive blocks below the 0.5 kJ mol^{-1} threshold. (C) Representative CV traces normalized to z-scores so different CV units are not mixed on one physical scale. The panel checks Leonardo's setup, plotting-stability tracking and plotting workflow on a compact molecule, not a biological protein-scale free-energy benchmark.

3.3.5 Trajectory and interaction analysis

The trajectory-analysis module is a Leonardo analysis pipeline for MD output. It uses MDTraj/NumPy for trajectory parsing and numerical operations, but Leonardo defines the contact-occupancy summaries, report structure and PyMOL-linked interpretation. For a residue-ligand contact c over T trajectory frames, contact occupancy is computed as

$$O_c = \frac{1}{T}\sum_{t=1}^{T} \mathbf{1}\left[d_c(t) < d_{\text{cut}}\right], \tag{5}$$

where $d_c(t)$ is the contact distance at frame t, d_{cut} is the selected interaction cutoff and $\mathbf{1}$ is an indicator function. The same framework is used for distance traces, RMSD

summaries and persistence reports. This module is used after MD to decide whether a docking interaction remains stable, whether a residue contact is intermittent or whether a ligand drifted away from the binding site.

3.3.6 Guided-exit binding diagnostic (historical ABFE-lite module)

ABFE-lite is the historical module name for Leonardo's resource-constrained guided-exit binding diagnostic workflow for local CPU studies. In this manuscript the module is reported as a guided-exit diagnostic, not as a production absolute binding free-energy method or a replacement for production alchemical ABFE campaigns, which require carefully designed thermodynamic cycles, restraints, standard-state corrections, uncertainty estimates and repeated sampling [44, 45]. The workflow constrains resource use, stages analysis and provides restraint diagnostics for preliminary local studies. The guided-exit option defines a cylinder from a binding-site point to an exit guide point. If $\mathbf{u}$ is the normalized cylinder axis and $\mathbf{r}$ is the ligand center-of-mass displacement from the binding-site reference, axial progress and radial displacement are

$$z = \mathbf{r} \cdot \mathbf{u}, \qquad \rho = \|\mathbf{r} - z\mathbf{u}\| . \tag{6}$$

The restraint penalizes ligand motion outside the desired exit channel,

$$U_{\mathrm{guide}}(\rho) = \frac{1}{2} k_\rho \max(0, \rho - \rho_0)^2, \tag{7}$$

so the ligand is encouraged to leave through the same entrance/exit path instead of escaping through arbitrary protein surfaces. The current manuscript treats the resulting windows as diagnostic state gates. Any state-ratio or volume-correction diagnostics emitted by the software are audit outputs from this guided-exit protocol, not production absolute binding free energies. The plotted occupancy profile is the dimensionless diagnostic

$$D_i = -\ln\left(\frac{n_i + 0.5}{\sum_j n_j + 0.5N}\right) - \min_k D_k,$$

where n_i is the sampled count in exit-coordinate bin i and N is the number of bins. It is a shifted histogram diagnostic, not a calibrated potential of mean force.

Here, the guided-exit diagnostic is evaluated as a setup, restraint-design and plotting workflow using the 3PTB/benzamidine system. A 5 ns guided-exit run generated 2500 sampled records at 2 ps spacing. The ligand center-of-mass exit coordinate reached 37.84 Å, the maximum-exit frame had a nearest protein-ligand distance of 29.00 Å and no close protein-ligand contacts within 4 Å, and the post-sampling gate accepted 340 bound hold-window records and 69 unbound hold-window records. These values check that Leonardo can generate, classify, analyze and plot a bound/unbound guided-exit trajectory. The manuscript diagnostic artifacts retain state-gating and trajectory fields only; no binding affinity, dissociation constant, association constant or IC50 is reported for this diagnostic case. The result is reported for workflow verification, not as a production ABFE benchmark, ligand-ranking result or binding-affinity estimate.

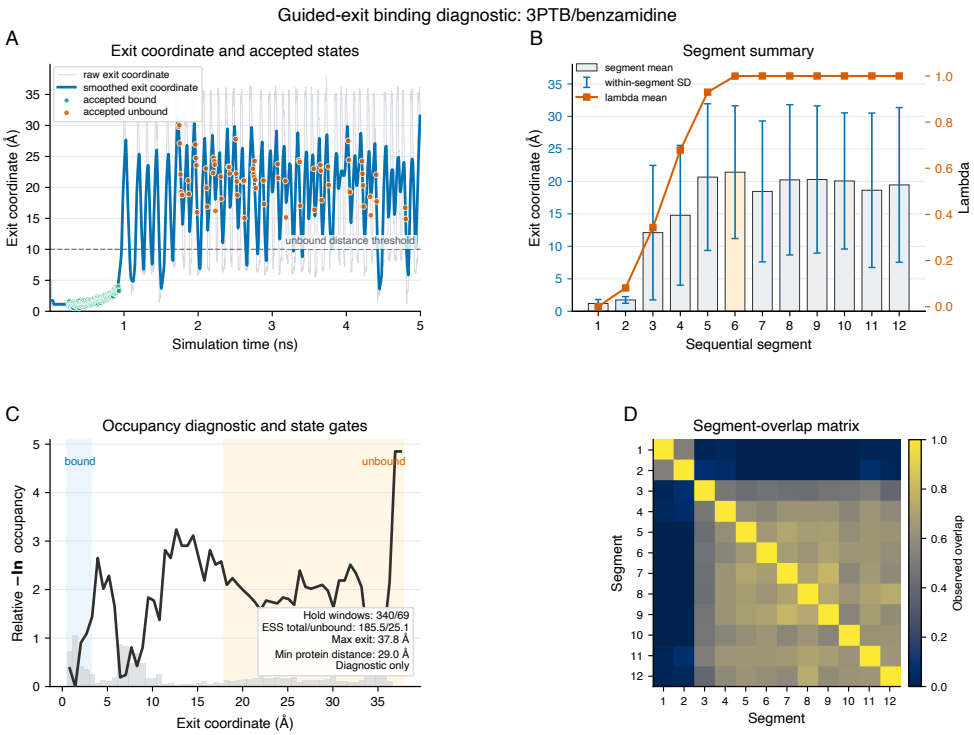


Fig. 11 Guided-exit binding diagnostic for the 3PTB/benzamidine system. The historical ABFE-lite module produced 2500 sampled records at 2 ps spacing, divided into 12 sequential analysis segments for plotting. (A) Real trajectory trace of the binding-site center-of-mass exit coordinate with the steering lambda schedule and state labels; a centered moving average is shown only to declutter the plotted trace. (B) Segment-level exit-coordinate means, standard deviations and lambda means. (C) Dimensionless shifted $-\ln$ occupancy histogram diagnostic from the sampled trajectory; the accepted diagnostic hold-window counts were 340/69 for bound/unbound states and ESS denotes effective sample size used only as a state-gating diagnostic. (D) Observed segment-overlap matrix from the real sampled exit-coordinate distributions. The panel reports a guided-exit workflow check rather than a production absolute binding-free-energy benchmark or binding-affinity estimate.

3.3.7 Psi4 quantum reaction pipeline

Leonardo includes Python-side reaction-path workflows for Psi4 and MOPAC. For Psi4, Leonardo generated QST2-like and QST3-like transition-state starting geometries for H + H_2 → H_2 + H, then delegated transition-state refinement, frequency analysis and IRC execution to Psi4. The QST2-like and QST3-like transition-state energies differed by only 0.000212 kcal/mol in this compact workflow case. The frequency analysis reported one imaginary mode with a primary imaginary frequency of 2982.405 cm^{-1}, and the resulting IRC profile had a 23.166 kcal/mol relative energy span. Leonardo's endpoint-connectivity check confirmed both forward and backward branches, matched 25 XYZ frames to 25 profile rows, placed the TS at the profile maximum and found the TS at least 23.166 kcal/mol above both IRC endpoints for this symmetric test.

For reaction-path workflows, Leonardo's QST2-like guess construction starts from mapped reactant and product geometries and constructs synchronous-transit guesses along

$$\mathbf{X}(\lambda) = (1-\lambda)\mathbf{X}_{\mathrm{R}} + \lambda\mathbf{X}_{\mathrm{P}}, \quad 0 \le \lambda \le 1. \tag{8}$$

The QST3-like variant incorporates an additional transition-state estimate $\mathbf{X}_{\mathrm{TS}}^{(0)}$ to bias the search near a chemically informed saddle region. A transition-state candidate is accepted only if frequency analysis identifies one dominant imaginary mode. Leonardo then requests, parses and assembles the engine IRC output corresponding to the descending reaction path in mass-weighted coordinates,

$$\frac{d\mathbf{s}}{d\tau} = \pm\frac{\mathbf{g}(\mathbf{s})}{\|\mathbf{g}(\mathbf{s})\|}, \tag{9}$$

where $\mathbf{g}$ is the mass-weighted gradient and the two signs define the reactant and product directions. The Psi4 example checks Leonardo's atom mapping, QST-like guess construction, Psi4 handoff, frequency parsing, endpoint-connectivity table generation, 25-point IRC assembly and PES plotting for a minimal three-atom reaction. It does not benchmark transition-state accuracy for general organic reactions or active-site models; users must still inspect the imaginary mode, endpoint connectivity, charge/multiplicity and method/basis suitability.

3.3.8 MOPAC capped-cluster reaction pipeline

The MOPAC module applies the same Leonardo reaction-path orchestration layer to semiempirical capped clusters. Leonardo selects a QM cluster around one or more reactive atoms, applies residue caps where needed, writes MOPAC inputs and maps user-facing QST2/QST3 workflow labels onto MOPAC-native SADDLE, TS, frequency and IRC stages. The external MOPAC engine provides the semiempirical energies and gradients [11, 46]; Leonardo provides cluster selection, cap bookkeeping, atom mapping, path construction and result interpretation.

For MOPAC, Leonardo used a 21-atom SN2-like capped cluster containing reactive F^-, CH_3Cl and fixed histidine/formamide mimics. The workflow used the PM7 semiempirical Hamiltonian [47], built QST-like transition-state candidates following synchronous-transit transition-state search concepts [38], checked one imaginary mode for the selected QST3-like transition state and generated a 25-point PM7 IRC/PES profile in mass-weighted reaction coordinates [39]. The primary imaginary frequency was 562.970 cm^{-1}, and the illustrative semiempirical cluster profile spanned 35.313 kcal/mol. The endpoint-connectivity table confirmed forward/backward branches, matching path frames and reactive-atom endpoint separation; the TS was at least 23.244 kcal/mol above both endpoints in this generated profile. The workflow case is a synthetic capped-cluster reaction-path example with fixed environment atoms, not a full QM/MM calculation or a protein-derived active-site extraction: it does not include protein electrostatics, solvation, entropy, higher-level QM comparison or enzymatic rate validation. This module is used for capped-cluster hypothesis prototyping where a compact semiempirical model is more feasible than a full ab initio model; PM7 has also been evaluated for enzyme catalytic-mechanism modeling in chymotrypsin [48].

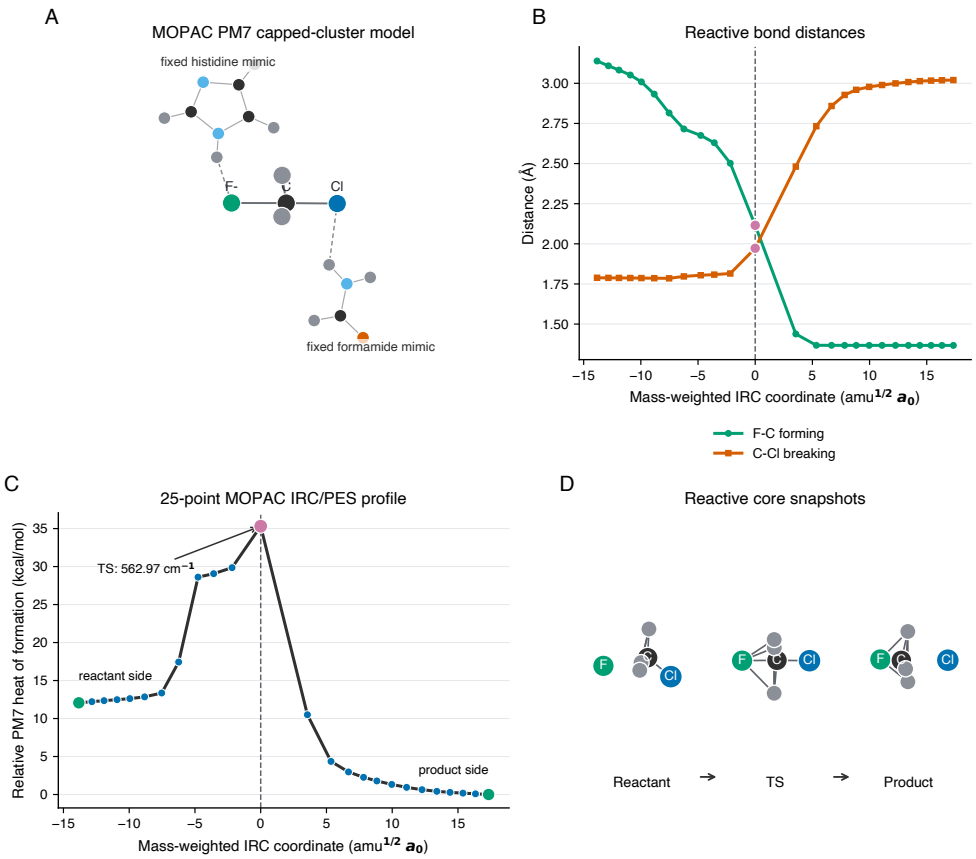


Fig. 12 MOPAC PM7 synthetic capped-cluster reaction-path example. MOPAC provides the semiempirical PM7 energies and gradients, while Leonardo contributes the object mapping, QST-like workflow labeling, transition-state check and plotting layer [38, 39, 46, 47]. (A) The 21-atom fixed capped-cluster model contains a reactive F^-/CH_3Cl core plus fixed histidine and formamide mimics. (B) Forming F–C and breaking C–Cl distances change along the mass-weighted IRC coordinate. (C) The selected QST3-like transition state has one dominant imaginary frequency and connects to a 25-point PM7 IRC/PES profile. (D) Reactant-side, transition-state and product-side reactive-core snapshots are exported from the generated IRC XYZ trajectory. This is a synthetic capped-cluster workflow demonstration, not a protein-derived QM/MM calculation or enzymatic-rate validation.

4 Validation scope and limitations

The workflow checks demonstrate that Leonardo can coordinate representative workflows and generate reproducible outputs, but they do not replace domain-specific benchmark studies for each scientific method. The interaction-analysis example checks parsing, residue-level contact grouping and conservative charge-aware reporting on one protein-ligand structure, and a three-complex ProLIF comparison provides external broad-class count context; broader atom-pair benchmarking against curated interaction annotations remains future work. The pocket-docking example checks fpocket parsing, grid construction and smina pose-summary reporting, not pocket-detection

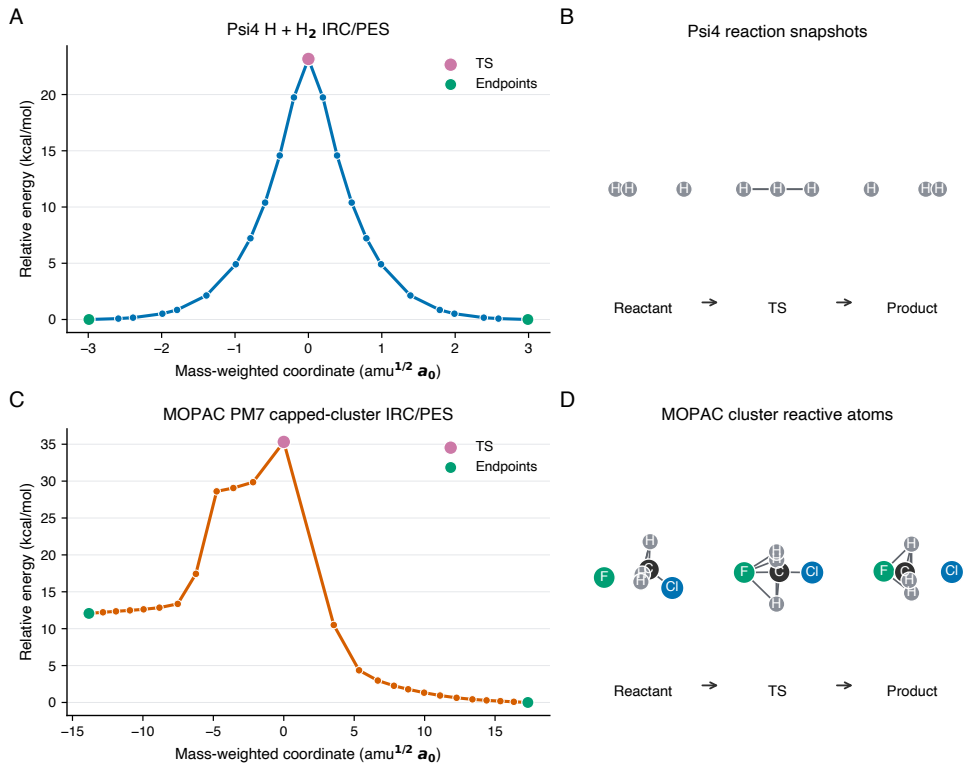


Fig. 13 Leonardo QST-like/IRC reaction-path workflow checks. (A) Psi4 H + H_2 IRC/PES profile for a three-atom case. (B) Psi4 reactant, transition-state and product snapshots from the generated IRC XYZ. (C) MOPAC PM7 capped-cluster IRC/PES profile for a 21-atom case. (D) MOPAC reactive atom snapshots for reactant, transition state and product. Both examples use 25 IRC points and check Leonardo's workflow orchestration, parsing and plotting rather than general transition-state accuracy.

or docking accuracy. The redocking example uses a well-behaved streptavidin/biotin case and repeated seeds; it supports the template-mapping and RMSD-reporting workflow but does not constitute a multi-target docking benchmark. The virtual-screening example checks ligand-library parsing and descriptor quality control, not enrichment performance, because no active/decoy benchmark set was included.

The OpenMM collective-variable example uses ACE-ALA-NME to exercise backend setup, metadynamics-style execution tracking and diagnostic-grid plotting on a compact molecule. It supports the software claim that Leonardo can execute and visualize CV workflows, but it should not be interpreted as a biological protein-scale free-energy benchmark or an uncertainty-calibrated comparison of force fields. Similarly, the guided-exit binding diagnostic is framed as a resource-limited workflow for preliminary studies; the 5 ns run checks guided-exit setup, trajectory analysis and state gating after both bound and unbound hold windows are populated. Production absolute binding free energies still require system-specific convergence checks,

careful restraints, repeated sampling, standard-state/restraint corrections and comparison with experimental or established computational references [44, 45]. The Psi4 and MOPAC examples check atom mapping, transition-state checks and IRC plotting on compact systems. The MOPAC example is a synthetic capped cluster with fixed histidine/formamide mimics, not a full protein-derived QM/MM calculation. These limitations define the manuscript's claim: Leonardo provides an auditable structural-bioinformatics workflow layer around established engines, while scientific accuracy remains dependent on the selected engine, input preparation, sampling design and system-specific assessment.

5 Conclusions

Leonardo provides a modular PyMOL-based platform for structural bioinformatics workflows that connect sequence analysis, structure preparation, protein-ligand interaction analysis, pocket docking, redocking, virtual screening, molecular dynamics, guided-exit binding diagnostics, quantum chemistry and semiempirical capped-cluster reaction-path analysis. Its contribution is the transparent workflow layer: object-aware graphical setup, documented defaults, project tracking, testing, reporting and plotting around established scientific engines. The evidence presented in this manuscript supports a software-platform claim focused on reproducible orchestration and backend verification; broader scientific benchmarks can be added module by module as community use cases expand.

List of abbreviations

ABFE: absolute binding free energy; CV: collective variable; FES: free-energy surface; IRC: intrinsic reaction coordinate; MSA: multiple sequence alignment; PDB: Protein Data Bank; PES: potential-energy surface; QM/MM: quantum mechanics/molecular mechanics; QST: quadratic synchronous transit; RMSD: root-mean-square deviation; TS: transition state.

Availability and requirements

- Project name: Leonardo.
- Project home page: https://github.com/juvenalyosa/Leonardo.
- Reviewer snapshot: GitHub tag `v0.2.0a3-reviewer`; a permanent public archive DOI remains pending for the exact journal-submission release.
- Peer-review access: private GitHub repository `juvenalyosa/Leonardo`; reviewer access can be provided by repository invitation or by a source ZIP supplied as confidential review material while the project remains private.
- Operating systems: workflow verification was performed on macOS; Linux installation scripts and environment specifications are distributed but still require clean-machine validation; Windows installation is provided through WSL2 helper scripts and remains pending independent validation.
- Programming language: Python.

- Installation: conda-based installation with an automated source-build and verification workflow.
- Core requirements: PyMOL, Qt/PySide or PyQt, NumPy, Matplotlib, RDKit and Biopython.
- Optional module-specific requirements: smina, fpocket, Meeko, Open Babel, AmberTools/OpenFF, OpenMM, PyMBAR, MDTraj, MODELLER, Psi4, MOPAC, AlphaFold/ColabFold, BLAST+, HMMER, MAFFT and Clustal Omega.
- License: MIT for Leonardo source code.
- Third-party licenses: summarized in `THIRD_PARTY_LICENSES.md`; module-specific engines may have independent licenses, citation requirements, registration requirements or academic/commercial restrictions.
- Restrictions to use by non-academics: none under the Leonardo MIT license; third-party engines may impose independent restrictions.

Declarations

Ethics approval and consent to participate

Not applicable.

Consent for publication

Not applicable.

Availability of data and materials

The source package, workflow-verification datasets, generated tables, test reports, figure-generation materials and checksum manifest are available in the private reviewer repository at https://github.com/juvenalyosa/Leonardo. The reviewer snapshot is identified by GitHub tag `v0.2.0a3-reviewer`. A permanent Zenodo DOI or equivalent archived release should be added for the exact public source package before journal submission.

Competing interests

JY is affiliated with Protyon Tech. MRG reports an external role as shareholder and CSO of Protyon BV. AJvdW declares no competing interests. The authors declare no other competing interests.

Funding

The authors received no specific funding for this work.

Authors' contributions

JY designed and implemented Leonardo, performed workflow-verification analyses, generated figures and wrote the manuscript. MRG contributed structural-biology

and protein-ligand-modeling guidance and reviewed the manuscript. AJvdW contributed clinical and translational-oncology perspective and reviewed the manuscript. All authors read and approved the final manuscript.

Acknowledgements

The authors acknowledge the developers and maintainers of the open-source scientific software cited in this work.

References

[1] Cock PJA, Antao T, Chang JT, Chapman BA, Cox CJ, Dalke A, et al. Biopython: freely available Python tools for computational molecular biology and bioinformatics. Bioinformatics. 2009;25(11):1422–1423. https://doi.org/10.1093/bioinformatics/btp163.

[2] RDKit Developers. RDKit: Open-source cheminformatics [Software DOI record]. Zenodo; 2026. Available from: https://doi.org/10.5281/zenodo.591637. Accessed 2026-07-02.

[3] O'Boyle NM, Banck M, James CA, Morley C, Vandermeersch T, Hutchison GR. Open Babel: An open chemical toolbox. Journal of Cheminformatics. 2011;3:33. https://doi.org/10.1186/1758-2946-3-33.

[4] Trott O, Olson AJ. AutoDock Vina: improving the speed and accuracy of docking with a new scoring function, efficient optimization, and multithreading. Journal of Computational Chemistry. 2010;31(2):455–461. https://doi.org/10.1002/jcc.21334.

[5] Koes DR, Baumgartner MP, Camacho CJ. Lessons Learned in Empirical Scoring with smina from the CSAR 2011 Benchmarking Exercise. Journal of Chemical Information and Modeling. 2013;53(8):1893–1904. https://doi.org/10.1021/ci300604z.

[6] Le Guilloux V, Schmidtke P, Tuffery P. Fpocket: an open source platform for ligand pocket detection. BMC Bioinformatics. 2009;10:168. https://doi.org/10.1186/1471-2105-10-168.

[7] Eastman P, Swails J, Chodera JD, McGibbon RT, Zhao Y, Beauchamp KA, et al. OpenMM 7: Rapid development of high performance algorithms for molecular dynamics. PLOS Computational Biology. 2017;13(7):e1005659. https://doi.org/10.1371/journal.pcbi.1005659.

[8] Shirts MR, Chodera JD. Statistically optimal analysis of samples from multiple equilibrium states. The Journal of Chemical Physics. 2008;129(12):124105. https://doi.org/10.1063/1.2978177.

[9] Sali A, Blundell TL. Comparative protein modelling by satisfaction of spatial restraints. Journal of Molecular Biology. 1993;234(3):779–815. https://doi.org/10.1006/jmbi.1993.1626.

[10] Smith DGA, Burns LA, Simmonett AC, Parrish RM, Schieber MC, Galvelis R, et al. PSI4 1.4: Open-source software for high-throughput quantum chemistry. The Journal of Chemical Physics. 2020;152(18):184108. https://doi.org/10.1063/5.0006002.

[11] Stewart JJP. MOPAC: A semiempirical molecular orbital program. Journal of Computer-Aided Molecular Design. 1990;4:1–103. https://doi.org/10.1007/BF00128336.

[12] Jumper J, Evans R, Pritzel A, Green T, Figurnov M, Ronneberger O, et al. Highly accurate protein structure prediction with AlphaFold. Nature. 2021;596:583–589. https://doi.org/10.1038/s41586-021-03819-2.

[13] Mirdita M, Schutze K, Moriwaki Y, Heo L, Ovchinnikov S, Steinegger M. ColabFold: making protein folding accessible to all. Nature Methods. 2022;19:679–682. https://doi.org/10.1038/s41592-022-01488-1.

[14] The UniProt Consortium. UniProt: the Universal Protein Knowledgebase in 2025. Nucleic Acids Research. 2025;53(D1):D609–D617. https://doi.org/10.1093/nar/gkae1010.

[15] Camacho C, Coulouris G, Avagyan V, Ma N, Papadopoulos J, Bealer K, et al. BLAST+: architecture and applications. BMC Bioinformatics. 2009;10:421. https://doi.org/10.1186/1471-2105-10-421.

[16] Eddy SR. Accelerated Profile HMM Searches. PLOS Computational Biology. 2011;7(10):e1002195. https://doi.org/10.1371/journal.pcbi.1002195.

[17] Katoh K, Standley DM. MAFFT Multiple Sequence Alignment Software Version 7: Improvements in Performance and Usability. Molecular Biology and Evolution. 2013;30(4):772–780. https://doi.org/10.1093/molbev/mst010.

[18] Sievers F, Wilm A, Dineen D, Gibson TJ, Karplus K, Li W, et al. Fast, scalable generation of high-quality protein multiple sequence alignments using Clustal Omega. Molecular Systems Biology. 2011;7:539. https://doi.org/10.1038/msb.2011.75.

[19] Schrödinger, Inc . Maestro: Complete modeling environment for molecular discovery [Product information]. Schrödinger, Inc.; 2026. Available from: https://www.schrodinger.com/platform/products/maestro/. Accessed 2026-07-02.

[20] Schrödinger, Inc . BioLuminate: Biologics modeling platform [Product information]. Schrödinger, Inc.; 2026. Available from: https://www.schrodinger.com/pl

atform/products/bioluminate/. Accessed 2026-07-02.

[21] Chemical Computing Group. Molecular Operating Environment: Integrated computer-aided molecular design platform [Product information]. Chemical Computing Group; 2026. Available from: https://www.chemcomp.com/en/Products.htm. Accessed 2026-07-02.

[22] Dassault Systèmes BIOVIA. BIOVIA Discovery Studio: Comprehensive modeling and simulation for life sciences research [Product information]. Dassault Systèmes BIOVIA; 2026. Available from: https://www.3ds.com/products/biovia/discovery-studio. Accessed 2026-07-02.

[23] Cambridge Crystallographic Data Centre. GOLD: Protein-ligand docking software [Product information]. Cambridge Crystallographic Data Centre; 2026. Available from: https://www.ccdc.cam.ac.uk/solutions/software/gold/. Accessed 2026-07-02.

[24] Schrödinger, LLC. The PyMOL Molecular Graphics System [Software citation]. Schrödinger, LLC; 2026. Available from: https://pymol.org/support.html. Accessed 2026-07-02.

[25] Pettersen EF, Goddard TD, Huang CC, Couch GS, Greenblatt DM, Meng EC, et al. UCSF Chimera–A visualization system for exploratory research and analysis. Journal of Computational Chemistry. 2004;25(13):1605–1612. https://doi.org/10.1002/jcc.20084.

[26] Pettersen EF, Goddard TD, Huang CC, Meng EC, Couch GS, Croll TI, et al. UCSF ChimeraX: Structure visualization for researchers, educators, and developers. Protein Science. 2021;30(1):70–82. https://doi.org/10.1002/pro.3943.

[27] Humphrey W, Dalke A, Schulten K. VMD: Visual molecular dynamics. Journal of Molecular Graphics. 1996;14(1):33–38. https://doi.org/10.1016/0263-7855(96)00018-5.

[28] Janson G, Zhang C, Prado MG, Paiardini A. PyMod 2.0: improvements in protein sequence-structure analysis and homology modeling within PyMOL. Bioinformatics. 2017;33(3):444–446. https://doi.org/10.1093/bioinformatics/btw638.

[29] Harris CR, Millman KJ, van der Walt SJ, Gommers R, Virtanen P, Cournapeau D, et al. Array programming with NumPy. Nature. 2020;585:357–362. https://doi.org/10.1038/s41586-020-2649-2.

[30] Hunter JD. Matplotlib: A 2D Graphics Environment. Computing in Science & Engineering. 2007;9(3):90–95. https://doi.org/10.1109/MCSE.2007.55.

[31] Forli Lab. Meeko: preparation of small molecules for AutoDock [Software repository]. GitHub; 2026. Available from: https://github.com/forlilab/Meeko. Accessed 2026-07-02.

[32] Mobley DL, Bannan CC, Rizzi A, Bayly CI, Chodera JD, Lim VT, et al. Escaping Atom Types in Force Fields Using Direct Chemical Perception. Journal of Chemical Theory and Computation. 2018;14(11):6076–6092. https://doi.org/10.1021/acs.jctc.8b00640.

[33] McGibbon RT, Beauchamp KA, Harrigan MP, Klein C, Swails JM, Hernandez CX, et al. MDTraj: A Modern Open Library for the Analysis of Molecular Dynamics Trajectories. Biophysical Journal. 2015;109(8):1528–1532. https://doi.org/10.1016/j.bpj.2015.08.015.

[34] Salentin S, Schreiber S, Haupt VJ, Adasme MF, Schroeder M. PLIP: fully automated protein-ligand interaction profiler. Nucleic Acids Research. 2015;43(W1):W443–W447. https://doi.org/10.1093/nar/gkv315.

[35] Bouysset C, Fiorucci S. ProLIF: a library to encode molecular interactions as fingerprints. Journal of Cheminformatics. 2021;13:72. https://doi.org/10.1186/s13321-021-00548-6.

[36] Berman HM, Westbrook J, Feng Z, Gilliland G, Bhat TN, Weissig H, et al. The Protein Data Bank. Nucleic Acids Research. 2000;28(1):235–242. https://doi.org/10.1093/nar/28.1.235.

[37] Amber Developers. AmberTools [Software documentation]. Amber Molecular Dynamics Package; 2026. Available from: https://ambermd.org/AmberTools.php. Accessed 2026-07-02.

[38] Peng C, Schlegel HB. Combining Synchronous Transit and Quasi-Newton Methods to Find Transition States. Israel Journal of Chemistry. 1993;33(4):449–454. https://doi.org/10.1002/ijch.199300051.

[39] Gonzalez C, Schlegel HB. Reaction path following in mass-weighted internal coordinates. The Journal of Physical Chemistry. 1990;94(14):5523–5527. https://doi.org/10.1021/j100377a021.

[40] Truchon JF, Bayly CI. Evaluating Virtual Screening Methods: Good and Bad Metrics for the Early Recognition Problem. Journal of Chemical Information and Modeling. 2007;47(2):488–508. https://doi.org/10.1021/ci600426e.

[41] Torrie GM, Valleau JP. Nonphysical sampling distributions in Monte Carlo free-energy estimation: Umbrella sampling. Journal of Computational Physics. 1977;23(2):187–199. https://doi.org/10.1016/0021-9991(77)90121-8.

[42] Kumar S, Rosenberg JM, Bouzida D, Swendsen RH, Kollman PA. The weighted histogram analysis method for free-energy calculations on biomolecules. I. The method. Journal of Computational Chemistry. 1992;13(8):1011–1021. https://doi.org/10.1002/jcc.540130812.

[43] Laio A, Parrinello M. Escaping free-energy minima. Proceedings of the National Academy of Sciences. 2002;99(20):12562–12566. https://doi.org/10.1073/pnas.202427399.

[44] Boresch S, Tettinger F, Leitgeb M, Karplus M. Absolute Binding Free Energies: A Quantitative Approach for Their Calculation. The Journal of Physical Chemistry B. 2003;107(35):9535–9551. https://doi.org/10.1021/jp0217839.

[45] Mey ASJS, Allen BK, Bruce Macdonald HE, Chodera JD, Hahn DF, Kuhn M, et al. Best Practices for Alchemical Free Energy Calculations. Living Journal of Computational Molecular Science. 2020;2(1):18378. https://doi.org/10.33011/livecoms.2.1.18378.

[46] Moussa JE, Stewart JJP. MOPAC: An open-source semiempirical molecular orbital program. Journal of Open Source Software. 2026;11(119):8025. https://doi.org/10.21105/joss.08025.

[47] Stewart JJP. Optimization of parameters for semiempirical methods VI: more modifications to the NDDO approximations and re-optimization of parameters. Journal of Molecular Modeling. 2013;19(1):1–32. https://doi.org/10.1007/s00894-012-1667-x.

[48] Stewart JJP. An investigation into the applicability of the semiempirical method PM7 for modeling the catalytic mechanism in the enzyme chymotrypsin. Journal of Molecular Modeling. 2017;23(5):154. https://doi.org/10.1007/s00894-017-3326-8.